Nombre: Jose Gerardo Ortiz Leñero	**Matrícula**: AL07182099
Nombre del curso: Gestión de proyectos de inteligencia artificial **Grupo:** CC.MSCA2102FLE.102.202630	**Nombre del profesor**: Ing. Tania Judith Mendoza Torres
Módulos: 15. Despliegue local. 16. Despliegue en la nube.	**Actividad**: Fase II. Documento de validación de pruebas del modelo DEMANDA-IA contenerizado.
Fecha: 01/Nov/2026	
MASTER INTELIGENCIA ARTIFICIAL	

Universidad Tecmilenio

Tabla de Contenido

1.- Objetivo y alcance **3**

2.- Entorno de prueba **3**

3.- Estrategia de pruebas **3**

4.- Validación del modelo **4**

5.- Pruebas funcionales de la API **7**

6.- Casos extremos **8**

7.- Análisis de sensibilidad del modelo **9**

8.- Integración, contenedor y desempeño **10**

9.- Hallazgos y acciones **17**

10.- Conclusiones **17**

Anexo **18**

Bibliografía ***19***

Universidad Tecmilenio

1.- Objetivo y alcance

Este documento reúne las pruebas que se hicieron a **DEMANDA-IA** antes de considerarlo listo para desplegar. Se plantearon cuatro preguntas concretas:

1. ¿El modelo pronostica mejor que las reglas simples que una farmacia podría usar sin IA?
2. ¿La API responde bien cuando recibe datos correctos y se defiende cuando recibe datos malos o raros?
3. ¿El frontend, el backend y el modelo se comunican como se diseñó?
4. ¿El contenedor arranca, se reporta sano y aguanta la carga de la corrida nocturna?

Las pruebas cubren el modelo, la API, la integración con la interfaz web, el contenedor y el desempeño. Quedan fuera la integración con un ERP real y las pruebas con ventas reales de la cadena, porque en el proyecto trabajamos con datos sintéticos.

2.- Entorno de prueba

Elemento	Detalle
Sistema operativo	Linux Ubuntu 24.04 (x86_64)
Python	3.12.3
Librerías	scikit-learn 1.8.0, FastAPI 0.162.0, Pydantic 2.14.1, Uvicorn 0.54.0, numpy 2.4.4, pandas 3.0.2 (solo entrenamiento), pytest 9.1.1
Pruebas automáticas y benchmark	Linux Ubuntu 24.04, Python 3.12.3, entorno virtual instalado solo con requirements.txt, réplica de la imagen con usuario sin privilegios, variable PORT y 2 workers
Ejecución en contenedor	Windows 11, Docker Desktop 4.97.0 (Docker Engine 29.9.2), imagen demanda-ia:1.0.0 (575.74 MB), 1 worker. Evidencia en Figuras 7 a 13
Ejecución en la nube	Google Cloud Run, región us-central1, revisión demanda-ia-00005-842, 1 vCPU, 1 GiB, simultaneidad 40, min-instances 0, max-instances 3. Evidencia en Figuras 14 a 16
Navegador	Capturas manuales

Tabla 1. Entorno donde se ejecutaron las pruebas.

3.- Estrategia de pruebas

Se organizaron las pruebas por nivel, de lo más interno (el modelo) a lo más externo (el contenedor publicado). Las automáticas están en la carpeta `tests/` y se corren con un solo comando. La prueba de humo y el benchmark tienen su script en `scripts/`; la integración con el navegador se documenta con capturas (Figuras 4 a 6).

Nivel	Qué se revisa	Herramienta	Casos
Modelo	Que las variables de entrenamiento y de producción sean idénticas, determinismo, desempeño contra baselines, calibración del P90 y comportamiento ante promociones, festivos, quincena y estacionalidad	pytest (test_modelo.py)	12
API funcional	Que cada endpoint responda con el contrato esperado	pytest + TestClient (test_api_funcional.py)	9

Casos extremos	Entradas inválidas, límites, valores atípicos y propiedades que deben cumplirse siempre	pytest (test_casos_extremos.py)	24
Integración frontend-backend	Flujo completo desde el navegador	Navegador (capturas manuales)	3 escenarios
Contenedor y humo	Arranque, healthcheck, 6 endpoints de la API, frontend, Swagger y un caso inválido sobre HTTP real	smoke_test.sh / .ps1	9 verificaciones
Desempeño	Latencia y tiempo de la corrida nocturna	benchmark.py	4 mediciones

Tabla 2. Niveles de prueba.

Como resultado global se puede concluir que las 45 pruebas automáticas pasaron en 2.2 segundos y la prueba de humo terminó sin fallas. La lista completa está en el anexo.

4.- Validación del modelo

Datos y método

El simulador generó dos años de ventas diarias (enero de 2024 a diciembre de 2025) para 50 sucursales y 30 SKUs de siete categorías. Se separaron los datos de dos formas para no obtener resultados engañosos:

- **Prueba temporal**. El modelo se entrenó con 40 sucursales hasta el 30 de septiembre de 2025 (733 200 filas) y se evaluó con octubre a diciembre de 2025 (110 400 filas). Así se simula el uso real: pronosticar días que el modelo todavía no ha visto.
- **Sucursales no vistas**. Diez sucursales nunca entraron al entrenamiento. Se evaluaron en el mismo trimestre (27 600 filas). Esto prueba si el modelo sirve para tiendas nuevas o para el resto de la red de 140.

Se hizo la comparación contra dos reglas que una farmacia usaría sin IA, la primera regla es repetir lo que se vendió el mismo día de la semana anterior (ingenuo lag 7) y la segunda es usar el promedio de los últimos 28 días.

También se agregó un tercer punto de referencia que llamo oráculo, el valor esperado que usa el propio simulador para generar las ventas. Ningún modelo real puede conocerlo, pero indica cuánto del error se debe al ruido aleatorio de los datos y no al modelo.

Métrica	Qué mide	Porqué se usa
WAPE	Suma de errores absolutos dividida entre la suma de ventas reales	Es la métrica más común en retail; no se rompe con días de venta cero, como sí lo hace el MAPE
MAE / RMSE	Error promedio en piezas, y versión que castiga más los errores grandes	Dan una idea del error en unidades
Sesgo	Suma de errores con signo entre suma de ventas	Indica si el modelo tiende a quedarse corto o pasarse
Cobertura P90	Porcentaje de días en que la venta real fue menor o igual al P90	Debe rondar 90 % para que el stock de seguridad sea confiable

Tabla 3. Métricas utilizadas.

Resultados generales

Indicador	Prueba temporal	Sucursales no vistas
WAPE diario modelo P50	46.2 %	45.8 %
WAPE diario ingenuo lag 7	66.9 %	65.7 %
WAPE diario media 28 días	49.0 %	48.1 %

WAPE diario oráculo (referencia: media real)	46.3 %	45.5 %
WAPE semanal modelo P50	20.1 %	21.5 %
WAPE semanal ingenuo lag 7	28.7 %	27.8 %
WAPE semanal oráculo	18.3 %	18.1 %
MAE diario (piezas)	3.38	3.68
RMSE diario (piezas)	5.36	6.71
Sesgo del P50	-11.4 %	-14.2 %
Cobertura P90	88.8 %	88.4 %

Tabla 4. Desempeño del modelo contra referencias.

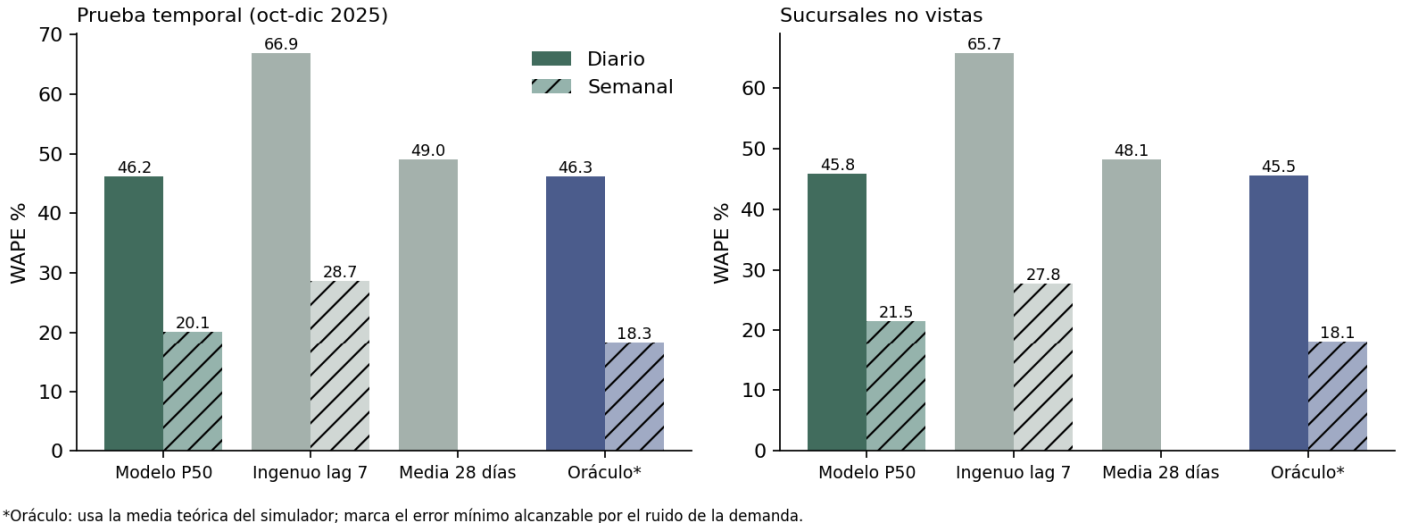


Figura 1. WAPE diario y semanal del modelo, las dos reglas simples y el oráculo.

La primera lectura de un WAPE diario de 46 % asusta. Cuando se pone junto al oráculo cambia el panorama, ya que un modelo que conociera perfectamente la demanda esperada tendría prácticamente el mismo error (46.3 %). Ese error no viene del modelo sino de que cada SKU vende pocas piezas por día en cada sucursal, y con volúmenes tan chicos el azar pesa mucho. El oráculo no es un piso estricto para el WAPE, porque el error absoluto se minimiza con la mediana y el oráculo usa la media; por eso el P50 puede quedar incluso ligeramente por debajo (46.2 % contra 46.3 %). Lo que sí muestra es que prácticamente todo el error restante es ruido.

A nivel semanal, que es el horizonte con el que se arma un pedido, el error baja a 20.1 %, contra 28.7 % de repetir la semana anterior. Es una reducción de 30 % en el error del pedido. La distancia con el oráculo semanal (18.3 %) es de menos de dos puntos, y es lo que queda por ganar con más variables o más datos.

Con las sucursales que el modelo nunca vio, el WAPE diario fue 45.8 % y el semanal 21.5 %, prácticamente igual que en la prueba temporal. Esto confirma que el modelo no memorizó tiendas, sino que aprendió de sus atributos y del historial reciente, así que se puede usar en las 140 sucursales, aunque se haya entrenado con 40.

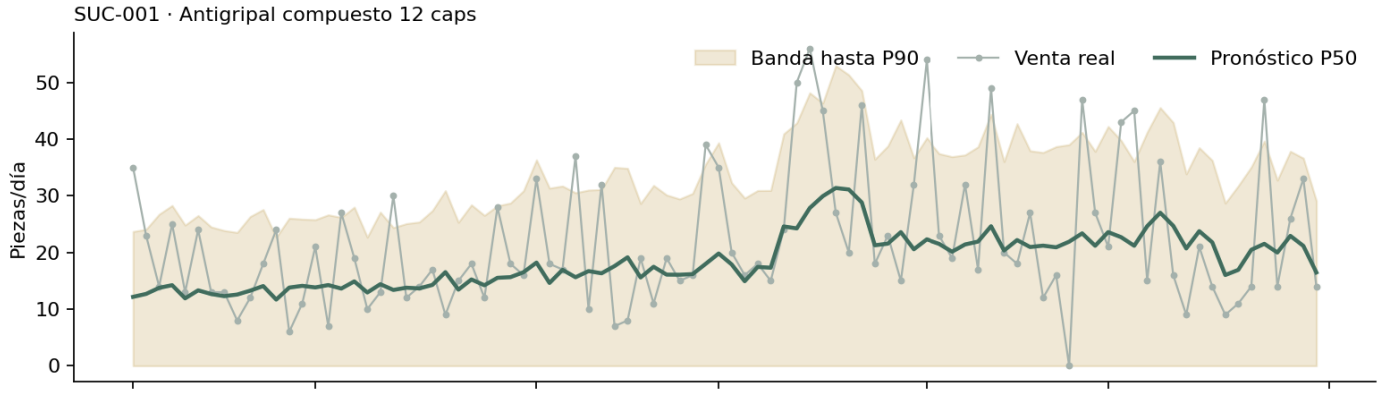


Figura 2. Venta real, pronóstico P50 y banda hasta P90 en SUC-001 durante el trimestre de prueba, para un antigripal

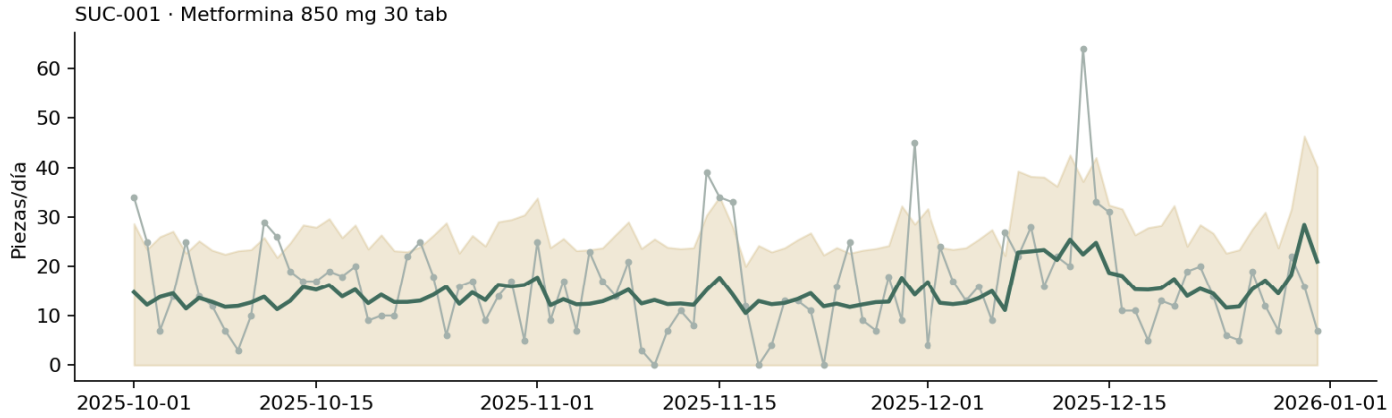


Figura 2a. Venta real, pronóstico P50 y banda hasta P90 en SUC-001 durante el trimestre de prueba, para metformina.

En la gráfica se ve lo que el modelo hace bien y lo que no puede hacer. Sigue el nivel y los cambios de régimen. El antigripal sube a finales de noviembre con el frío y el P50 sube con él. La metformina tiene saltos en las quincenas. Lo que no intenta es perseguir cada pico individual, porque esos picos son ruido. La banda P90 cubre la mayoría de los días altos, que es justo su función.

Resultados por categoría

Categoría	WAPE modelo	WAPE lag 7	Mejora	Cobertura P90
Analgésicos	43.6 %	63.1 %	19.5 pts	88.5 %
Antigripales	44.0 %	62.7 %	18.7 pts	88.1 %
Sueros y antidiarreicos	46.7 %	68.5 %	21.9 pts	89.9 %
Crónicos	46.1 %	68.8 %	22.7 pts	88.9 %
Vitaminas	50.0 %	71.7 %	21.7 pts	88.7 %
Dermocosmética	53.1 %	76.2 %	23.0 pts	89.5 %
Infantil	50.0 %	71.4 %	21.4 pts	88.4 %

Tabla 5. Desempeño por categoría (prueba temporal).

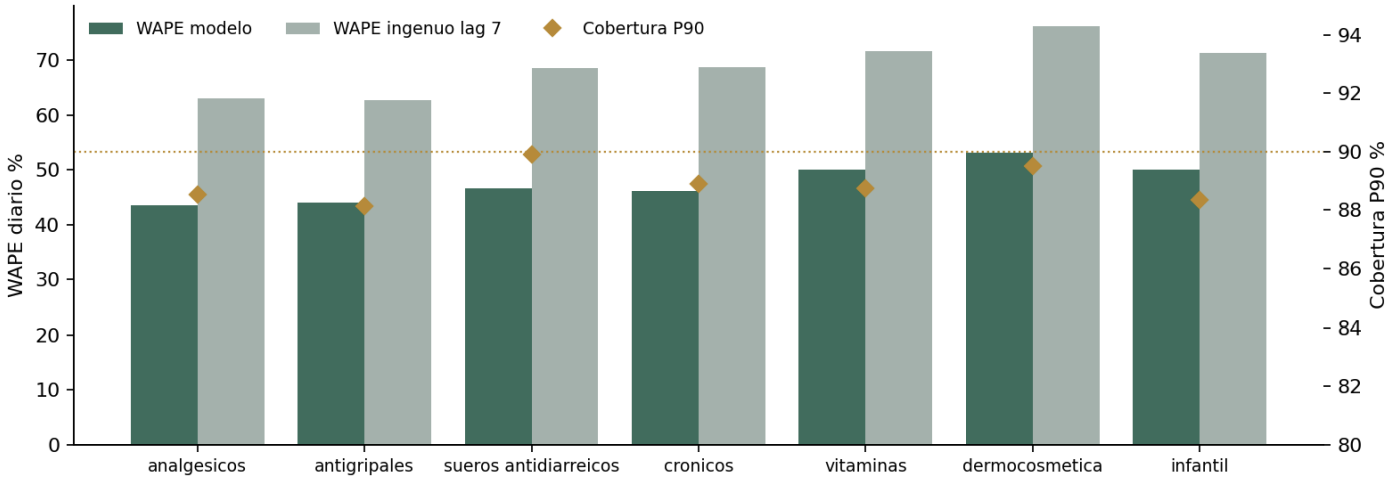


Figura 3. WAPE diario por categoría y cobertura del P90 (línea punteada en 90 %).

El modelo mejora a la regla simple en todas las categorías, entre 19 y 23 puntos. Las que peor salen son dermocosmética, vitaminas e infantil. Tienen en común que son productos de menor rotación diaria (bloqueador, multivitamínico, fórmula láctea), y con menos piezas por día el ruido relativo es mayor. Para esas categorías tendría más sentido pronosticar por semana que por día.

Calibración del P90 y sesgo

La cobertura del P90 quedó entre 87.5 % y 90.6 % en todas las categorías y en ambos conjuntos. Está ligeramente por debajo del 90 % nominal, algo normal en regresión por cuantiles con datos de conteo. Para el negocio significa que, en uno de cada diez días, más o menos, la venta supera el escenario alto, y el stock de seguridad está diseñado sabiendo eso.

El hallazgo que más me importa reportar es el sesgo de −11.4 % del P50. No es un error de programación. El P50 estima la mediana y en ventas al menudeo la distribución está cargada a la derecha (muchos días normales y algunos muy altos), así que la mediana queda por debajo del promedio. Si alguien sumara solo los P50 de la semana para hacer el pedido, pediría de menos. Por eso el endpoint de reabasto nunca usa solo el P50. Suma un stock de seguridad calculado con la distancia al P90. En la siguiente versión propongo agregar un tercer modelo que estime la media (pérdida Poisson) para la demanda esperada y dejar los cuantiles solo para el colchón.

5.- Pruebas funcionales de la API

Cada prueba levanta la aplicación completa en memoria con el TestClient de FastAPI, carga el modelo real y hace peticiones HTTP como lo haría un cliente.

Prueba	Petición	Resultado esperado	Resultado obtenido
Salud del servicio	GET /api/v1/salud	200, estado ok y modelo cargado	Pasó
Metadatos	GET /api/v1/modelo	15 variables y evaluación en los dos conjuntos	Pasó
Catálogo	GET /api/v1/catalogo	140 sucursales, 30 SKUs, límite de 1000 por lote	Pasó
Historial	GET /api/v1/historial	28 días que terminan en la fecha pedida	Pasó
Pronóstico puntual	POST /api/v1/pronostico	0 ≤ P50 ≤ P90; unidades sugeridas ≥ P90; encabezado de latencia	Pasó
Lote	POST /api/v1/pronostico/lote con 30 SKUs	30 resultados en el mismo orden de entrada	Pasó

Reabasto	POST /api/v1/reabasto a 7 días	7 días; pedido = techo (demanda + seguridad − existencia)	Pasó
Frontend servido	GET / y GET /app.js	200 y la página contiene DEMANDA-IA	Pasó
Documentación	GET /openapi.json	Incluye los endpoints de inferencia y salud	Pasó

Tabla 6. Pruebas funcionales por endpoint.

Un ejemplo real de respuesta del endpoint de reabasto, obtenido en la prueba de humo contra el servicio levantado:

```
POST /api/v1/reabasto  (SUC-001, SKU-0009 suero oral, 7 días, existencia 15)
{
  "dias": [{"fecha":"2026-11-02","p50":20.61,"p90":31.54}, ... 7 días ...],
  "demanda_esperada": 132.26,
  "stock_seguridad": 42.73,
  "existencia_actual": 15.0,
  "pedido_sugerido": 160,
  "advertencias": [],
  "modelo_version": "1.0.0"
}
```

La cuenta es directa: 132.26 de demanda esperada más 42.73 de seguridad menos 15 en anaquel da 159.99, que se redondea hacia arriba a 160 piezas.

6.- Casos extremos

Frente a cualquier entrada, la API debe contestar algo útil, se aplicó un criterio simple. Si la entrada es inválida, un código 422 que diga qué campo falló. Si es válida pero rara, un pronóstico razonable más una advertencia. Nunca un error 500 ni un pronóstico negativo.

Caso	Entrada	Esperado	Obtenido
Sucursal inexistente	SUC-999	422	422, sucursal desconocida
SKU inexistente	SKU-9999	422	422, SKU desconocido
Venta negativa	lag_7=-1	422	422, debe ser ≥ 0
Tipo incorrecto	media_28 = "muchas"	422	422, no es número
Fecha fuera de rango	2031-01-01	422	422, rango soportado
Fecha imposible	2026-02-30	422	422, fecha inválida
Precio cero	precio = 0	422	422, debe ser > 0
Campo no previsto	campo_extra = 1	422	422, campo no permitido
Campo faltante	sin media_7	422	422
JSON mal formado	{sucursal: SUC-001	422	422
IDs en minúsculas y con espacios	" suc-001 ", "sku-0001"	Se normalizan y responde 200	200, SUC-001
Historial en ceros	lag_7 = media_7 = media_28 = 0	Pronóstico ≥ 0 y advertencia	P50 1.2, aviso de SKU nuevo o quiebre
Pico atípico	historial de 5 000 piezas/día	Pronóstico finito y advertencia	P50 30.7, aviso de ventas atípicas
Precio de 1 peso o de 500 pesos	fuera del rango entrenado	Se acota y avisa	Aviso de precio relativo fuera de rango

Lote vacío / lote de 1001	0 y 1001 registros	422	422 en ambos
Lote con un registro malo	1 válido + 1 con SKU inválido	422 de todo el lote	422, señala registros.1.sku
Horizonte 0 o 15 días	límite 1 a 14	422	422 en ambos
Historial corto o negativo	27 días; o un valor −3	422	422 en ambos
Existencia enorme	100 000 piezas en anaquel	Pedido 0	Pedido 0
Horizonte máximo con cambio de año	14 días desde 2026-12-20	Termina 2027-01-02	Correcto
Historial fuera del periodo simulado	hasta = 2028-03-01	422	422
Prueba de propiedad	2 000 solicitudes aleatorias válidas	Nunca negativo; P90 ≥ P50 siempre	Se cumplió en las 2 000

Tabla 7. Casos extremos evaluados.

De estas pruebas, salieron dos decisiones de diseño. La primera, un lote con un solo registro inválido se rechaza completo, en lugar de procesar los válidos y omitir el malo. Este fue seleccionado porque en la corrida nocturna es más seguro que el ERP sepa que algo falló a que reciba un pedido incompleto sin darse cuenta. La segunda, cuando el precio queda fuera de lo que el modelo conoce, la API no inventa una respuesta. Acota el precio al rango aprendido y lo dice en el campo advertencias.

7.- Análisis de sensibilidad del modelo

Además de medir el error, quería ver si el modelo reacciona a las variables de negocio en la dirección correcta. Fijé un caso base (un antigripal en SUC-001 con historial de 10 piezas diarias) y cambié una cosa a la vez.

Escenario	P50	P90	Advertencia
Base: antigripal, miércoles 4-nov-2026, historial 10 u/día	11.79	22.70	Ninguna
Promoción con 15 % de descuento	17.37	31.70	Ninguna
Precio 20 % arriba de lista	11.79	22.70	Precio fuera de rango, se acota
Invierno: miércoles 13-ene-2027	11.46	22.30	Ninguna
Verano: miércoles 15-jul-2026	7.60	15.89	Ninguna
Festivo: lunes 16-nov-2026 (Revolución)	8.82	20.04	Ninguna
Lunes normal 9-nov-2026	12.16	23.52	Ninguna
Historial en ceros (SKU nuevo o quiebre)	1.20	4.05	Historial en ceros
Pico atípico: historial 5 000 u/día	30.67	48.82	Ventas atípicas, revisar
Precio extremo: 1 peso	15.46	28.85	Precio fuera de rango, se acota
Crónico (metformina) día 15, quincena	11.11	21.61	Ninguna
Crónico (metformina) día 8	8.98	16.93	Ninguna

Tabla 8. Respuesta del modelo a escenarios de negocio.

Casi todo se mueve como se espera. La promoción con 15 % de descuento sube el P50 de 11.8 a 17.4 piezas (+47 %), en la dirección y magnitud esperadas; el simulador implica ≈ +60 % al combinar el efecto de la promoción (×1.25) con la elasticidad al precio (×1.28), así que el modelo captura la mayor parte del efecto, pero lo subestima en parte.

El festivo del 16 de noviembre baja el pronóstico a 8.8 contra 12.2 de un lunes normal. La metformina sube de 9.0 a 11.1 en día de quincena. El verano baja el antigripal a 7.6.

Hay tres resultados que vale la pena explicar porque podrían parecer errores:

- **Precio 20 % arriba de lista: sin cambio.** El modelo nunca vio precios por encima del de lista, solo descuentos. Un árbol de decisión no extrapola, así que trata cualquier precio alto igual que el precio de lista. Esto se detectó en este análisis y por eso la API ahora avisa cuando el precio relativo sale del rango 0.75 a 1.05.
- **Invierno casi igual que noviembre (11.5 contra 11.8).** No es que el modelo ignore la temporada. Al fijar el historial en 10 piezas diarias, le estoy diciendo que en enero se vendió lo mismo que en noviembre, y el modelo le cree más al historial reciente que al calendario. En la operación real el historial de enero ya traería el alza.
- **Pico atípico de 5 000 piezas: P50 de 30.7.** El modelo no sigue el pico porque nunca vio algo así; se queda en el máximo que conoce. En este contexto es el comportamiento deseable, porque un pico así suele ser una venta institucional o un error de captura. La advertencia pide revisarlo a mano.

8.- Integración, contenedor y desempeño

Integración frontend-backend

Todo lo calcula el backend, la interfaz web no tiene lógica de negocio. Al presionar «Calcular pedido», el navegador hace dos llamadas, primero `GET /api/v1/historial` para traer los 28 días previos y después `POST /api/v1/reabasto` con ese historial, la existencia y el horizonte. Probé tres escenarios con un navegador automatizado:

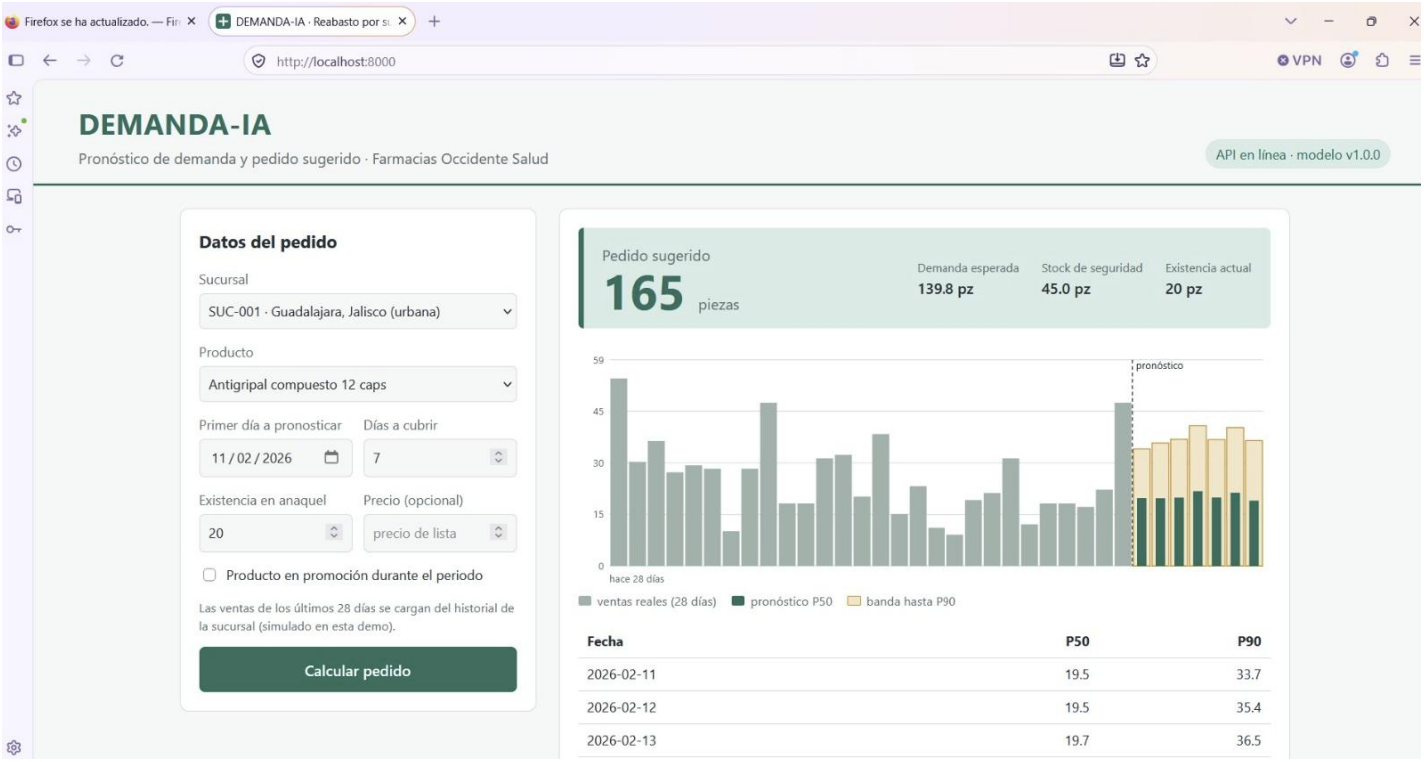


Figura 4. Escenario normal: el pedido sugerido (165 pz) se desglosa en demanda esperada, stock de seguridad y existencia.

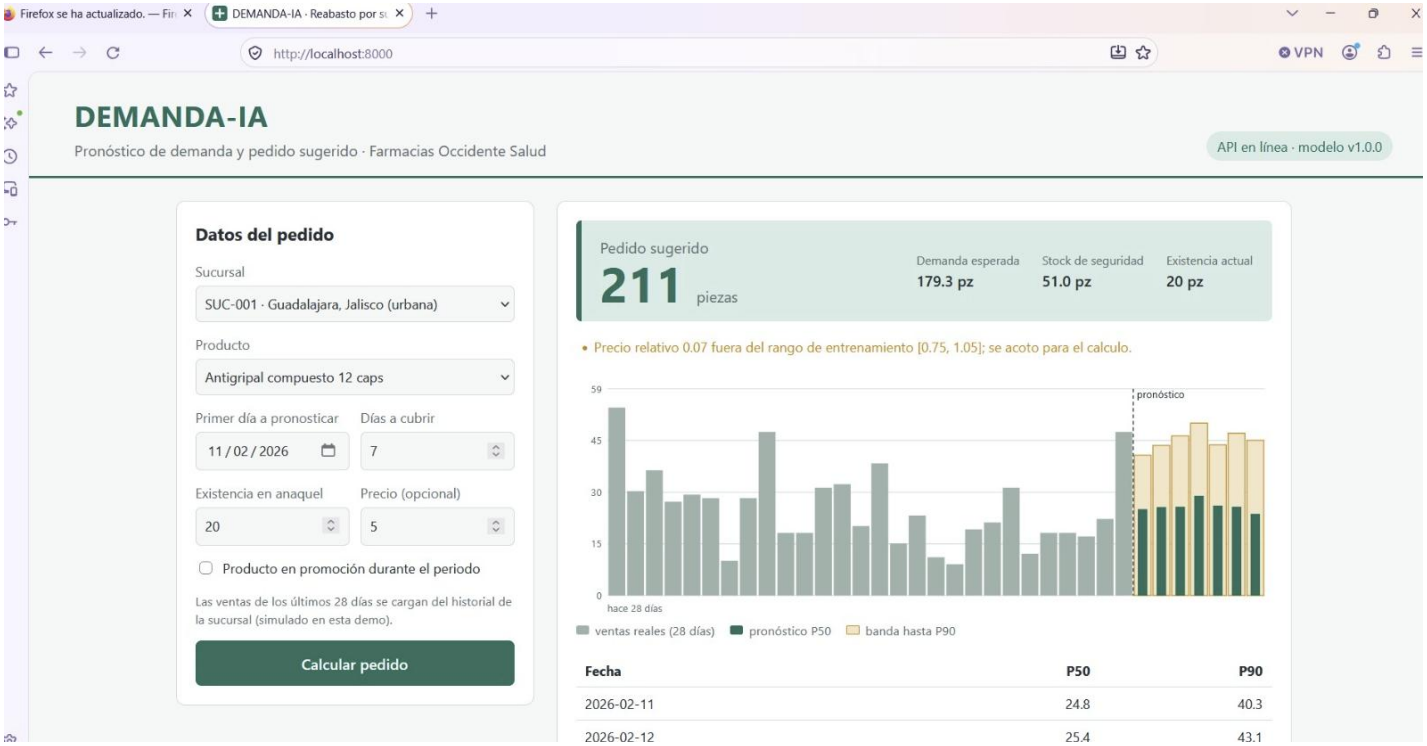


Figura 5. Escenario con precio de 5 pesos: el backend acota el precio y la interfaz muestra la advertencia que devuelve la API.

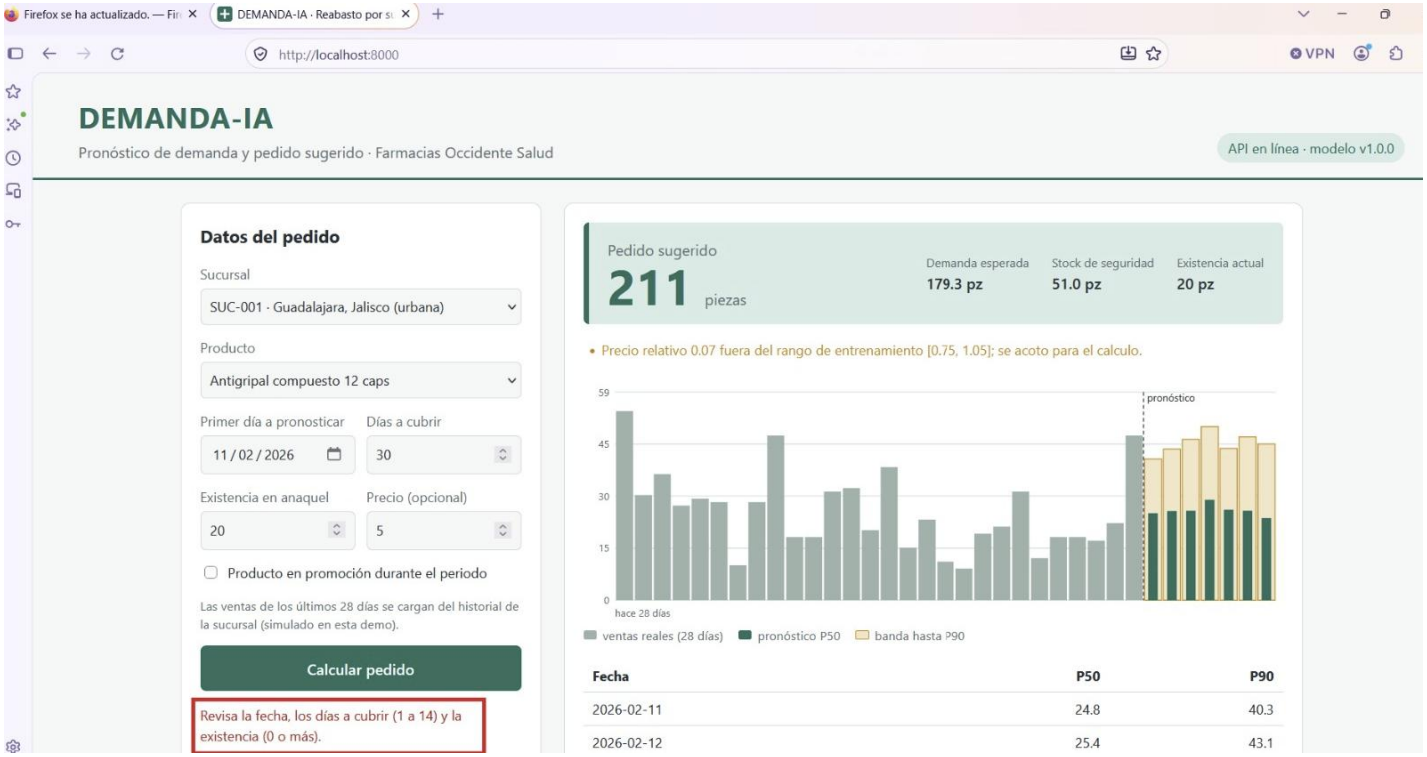


Figura 6. Escenario inválido (30 días a cubrir): la interfaz valida antes de llamar a la API y explica cómo corregir.

Ejecución tipo contenedor y prueba de humo

El servicio arrancó con el usuario sin privilegios, dos workers y el puerto tomado de la variable PORT. Cada worker cargó el modelo en unos 2 segundos. El comando del HEALTHCHECK del Dockerfile respondió correctamente y la prueba de humo pasó completa. Salida de smoke_test.sh (Linux, macOS o Git Bash):

```
OK    [200] GET  /api/v1/salud
OK    [200] GET  /api/v1/modelo
OK    [200] GET  /api/v1/catalogo
OK    [200] GET  / (frontend)
OK    [200] GET  /docs (Swagger)
OK    [200] POST /api/v1/pronostico
OK    [200] POST /api/v1/pronostico/lote
OK    [200] POST /api/v1/reabasto
OK    [422] POST pronostico SKU invalido
-----
Prueba de humo superada
```

Evidencia con Docker

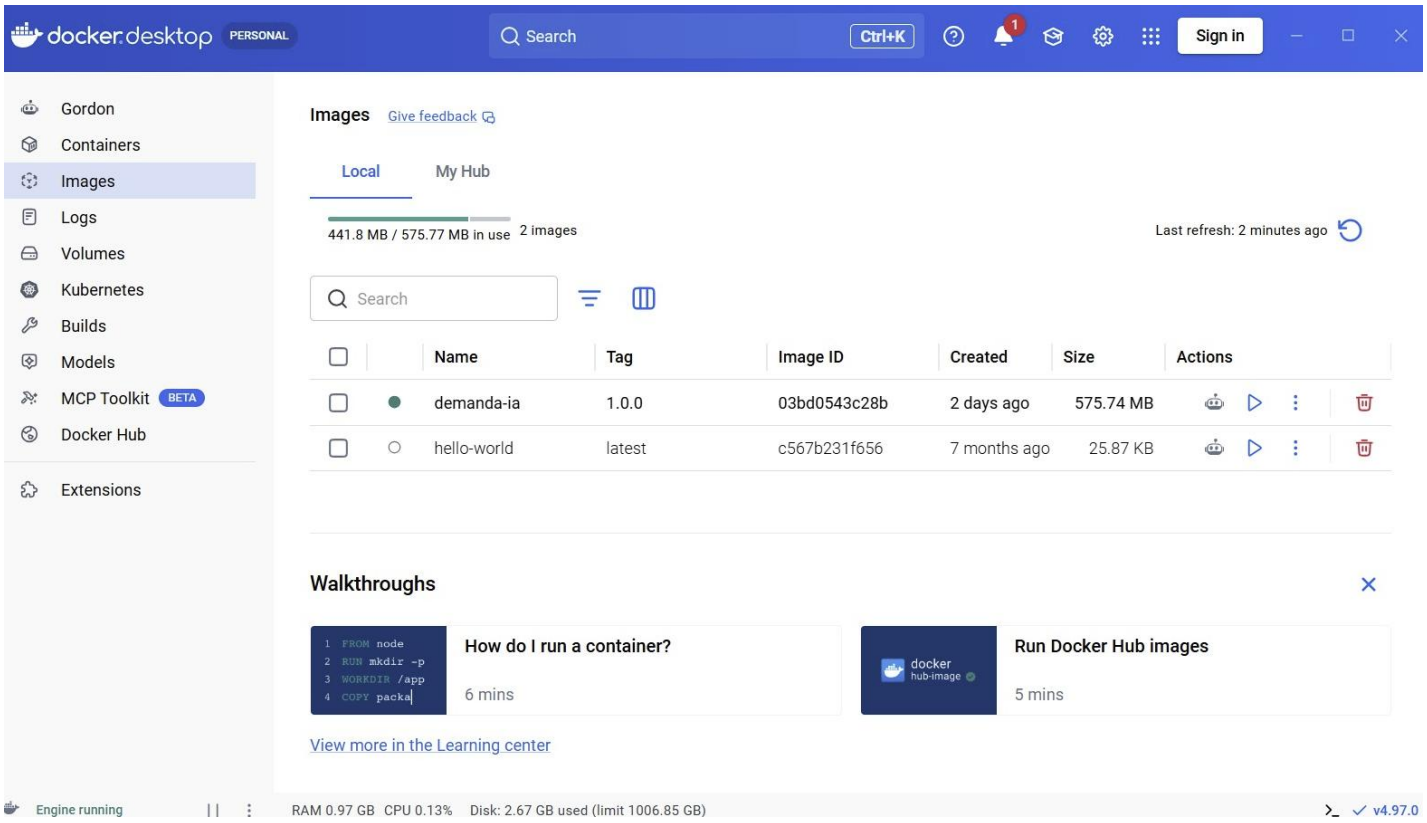


Figura 7. Docker desktop con la imagen de demanda-ia:1.0.0.



Figura 8. Salida final de `docker build -t demanda-ia:1.0.0 .`

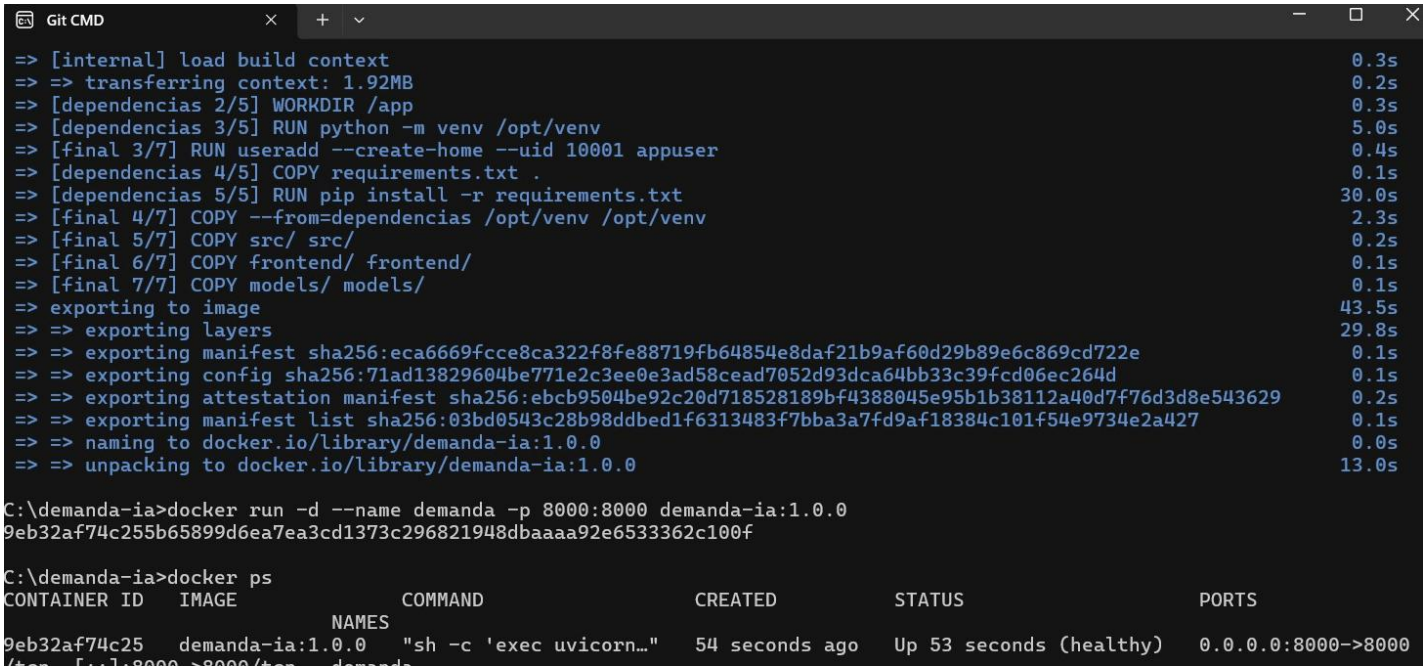


Figura 9. `docker ps` *mostrando el contenedor en estado (healthy)*

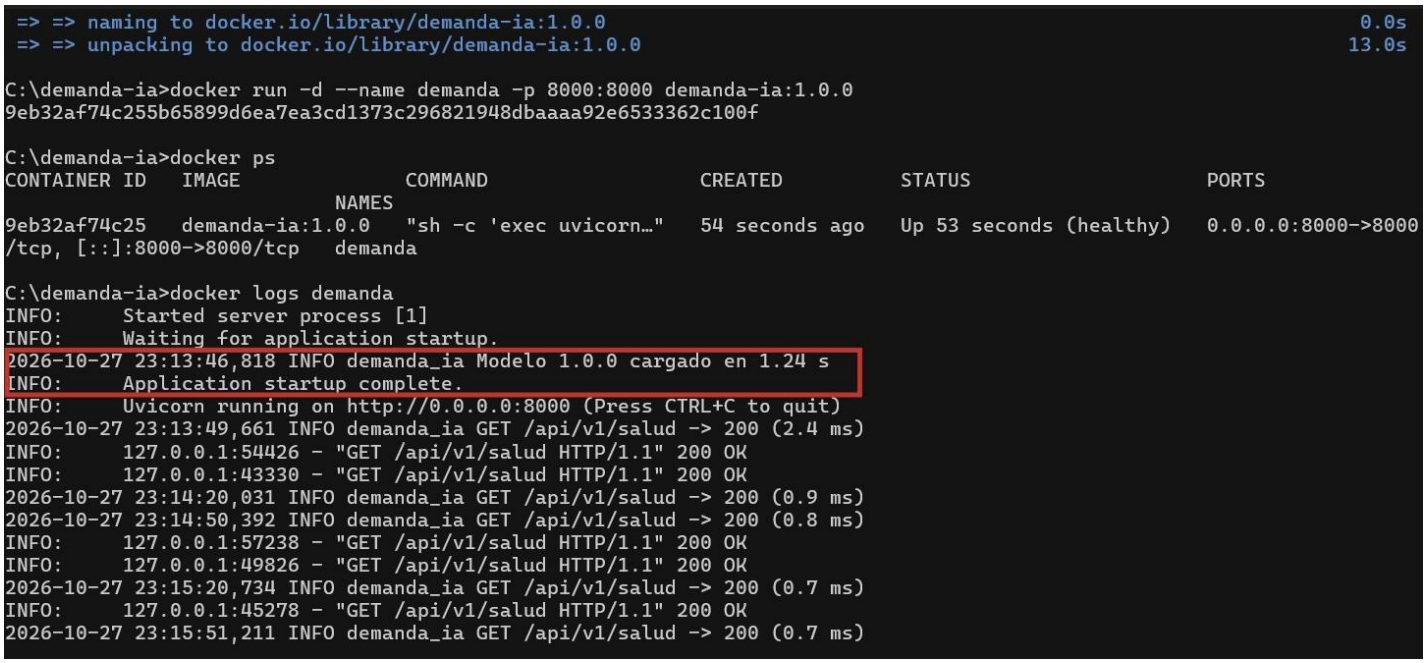


Figura 10. `docker logs demanda` *con la línea «Modelo 1.0.0 cargado»*

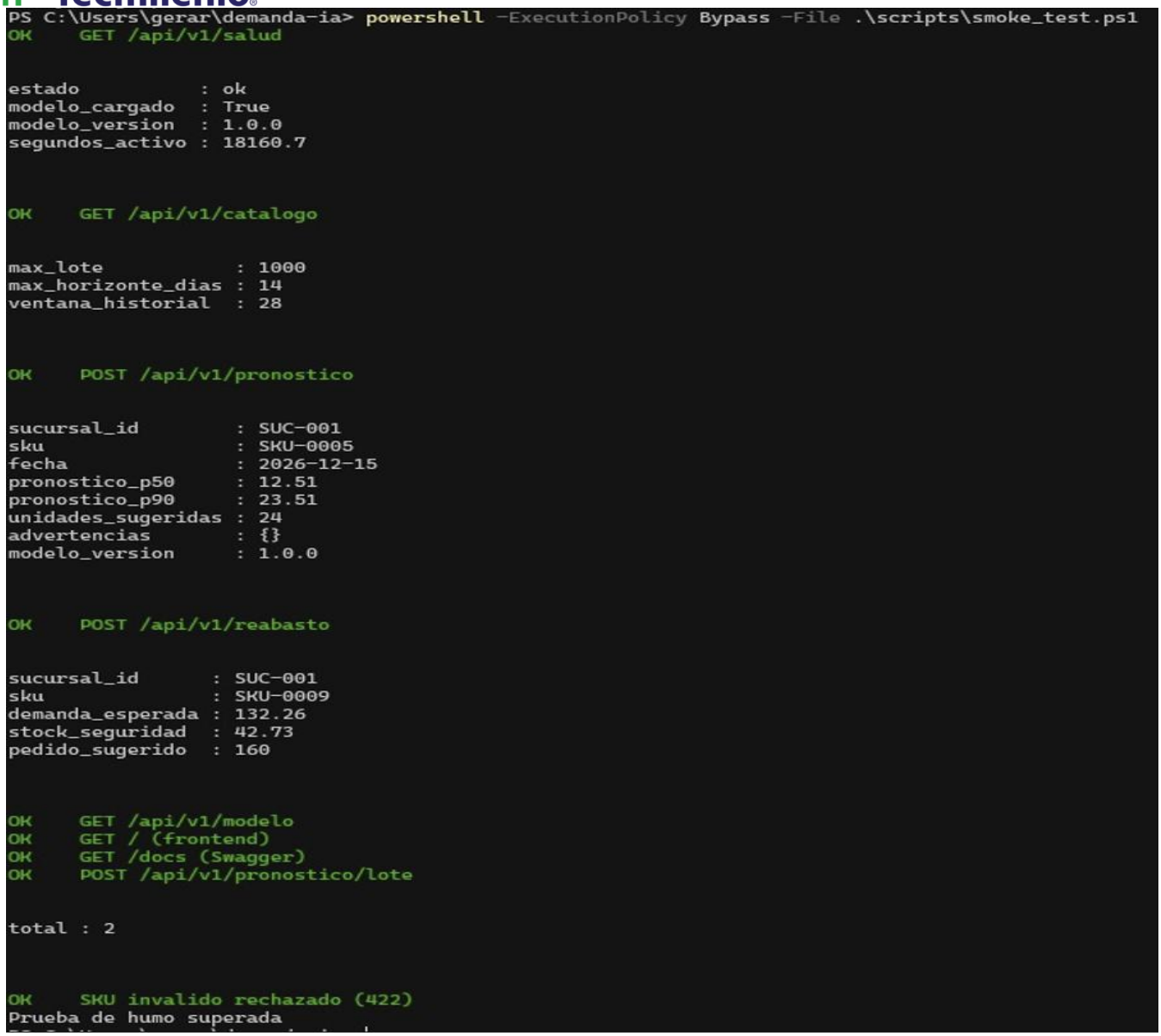


Figura 11. Prueba de humo con smoke_test.ps1 en Windows PowerShell contra el contenedor demanda-ia:1.0.0: 9 verificaciones superadas.

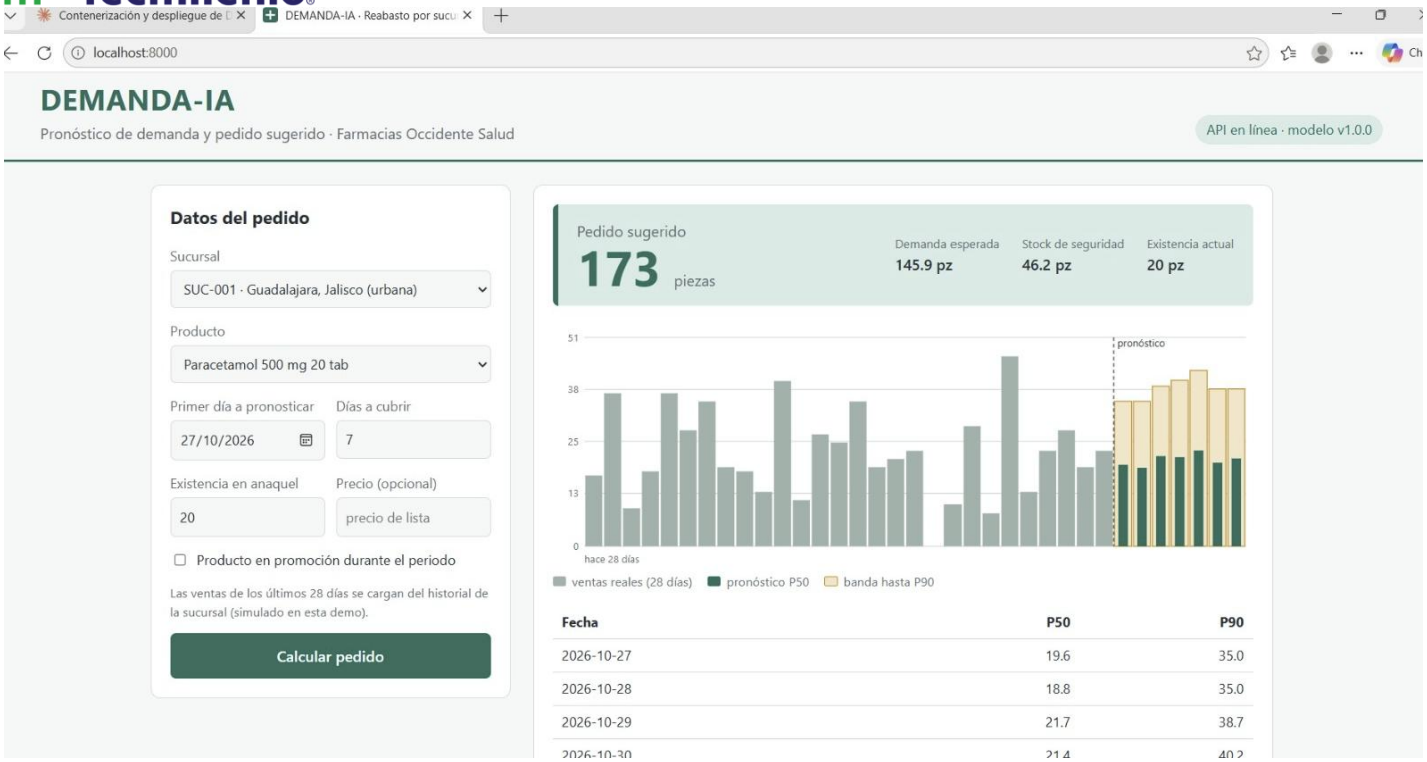


Figura 12. Interfaz web en http://localhost:8000/

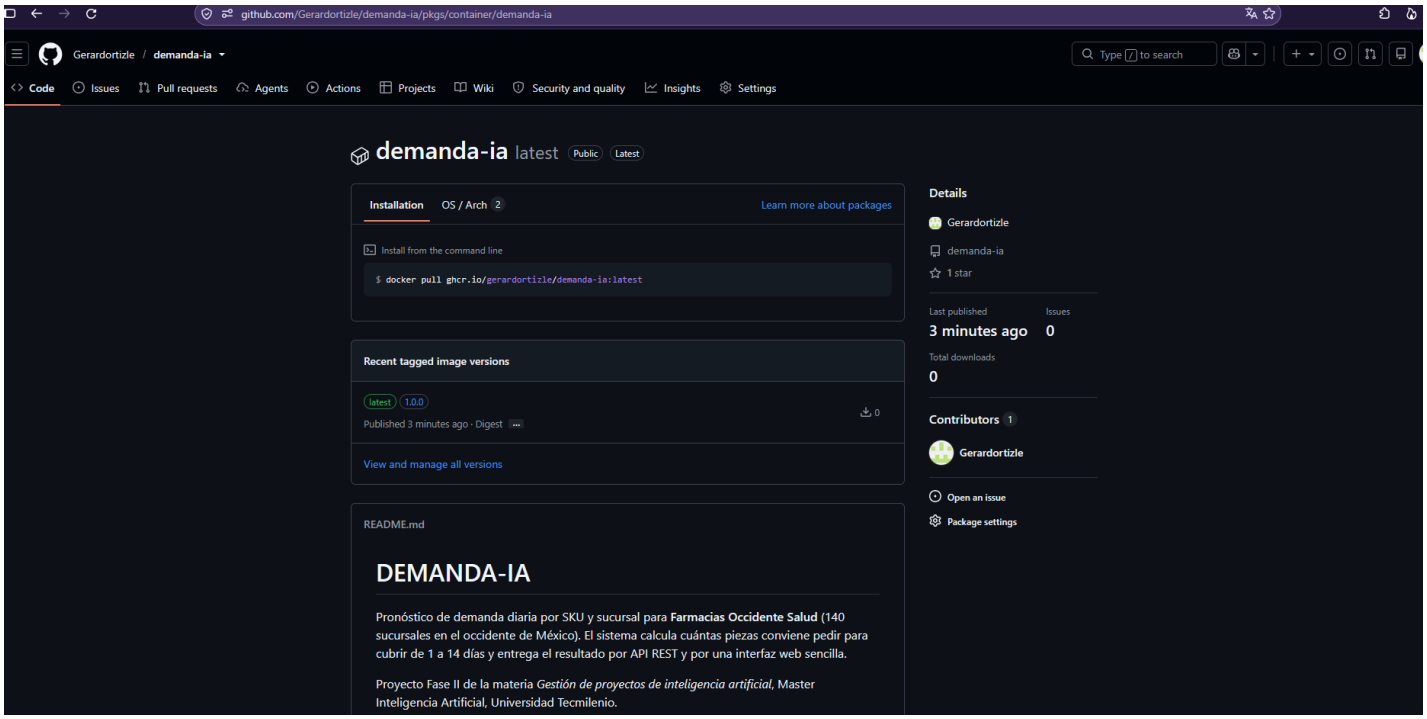


Figura 13. Imagen demanda-ia publicada por el pipeline de CI en GitHub Container Registry con las etiquetas 1.0.0 y latest.

Evidencias en Google Cloud Run

La misma imagen publicada por el CI se desplegó en Google Cloud Run siguiendo la sección 7 del manual de despliegue. Sobre la URL pública https://demanda-ia-874563229975.us-central1.run.app se repitió la prueba de humo con las mismas 9 verificaciones que en local. El pedido sugerido para suero oral en SUC-001 (7 días, existencia 15) fue de 160 piezas, idéntico al del contenedor local, lo que confirma que la imagen desplegada es la misma que se validó.

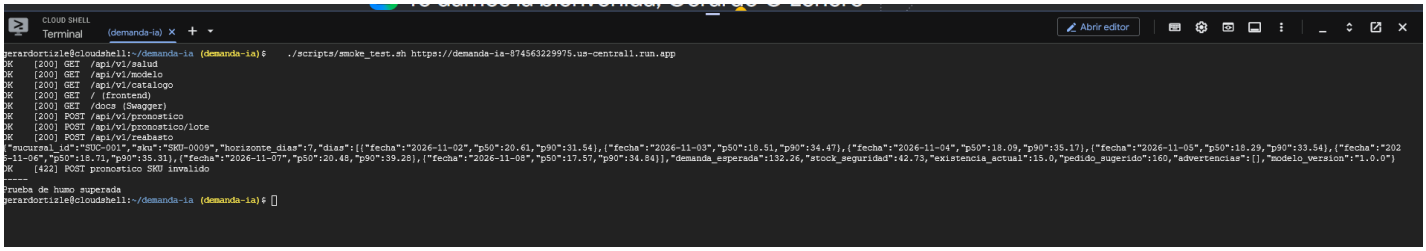


Figura 14. Prueba de humo con smoke_test.sh contra la URL de Cloud Run desde Cloud Shell: 9 verificaciones superadas.

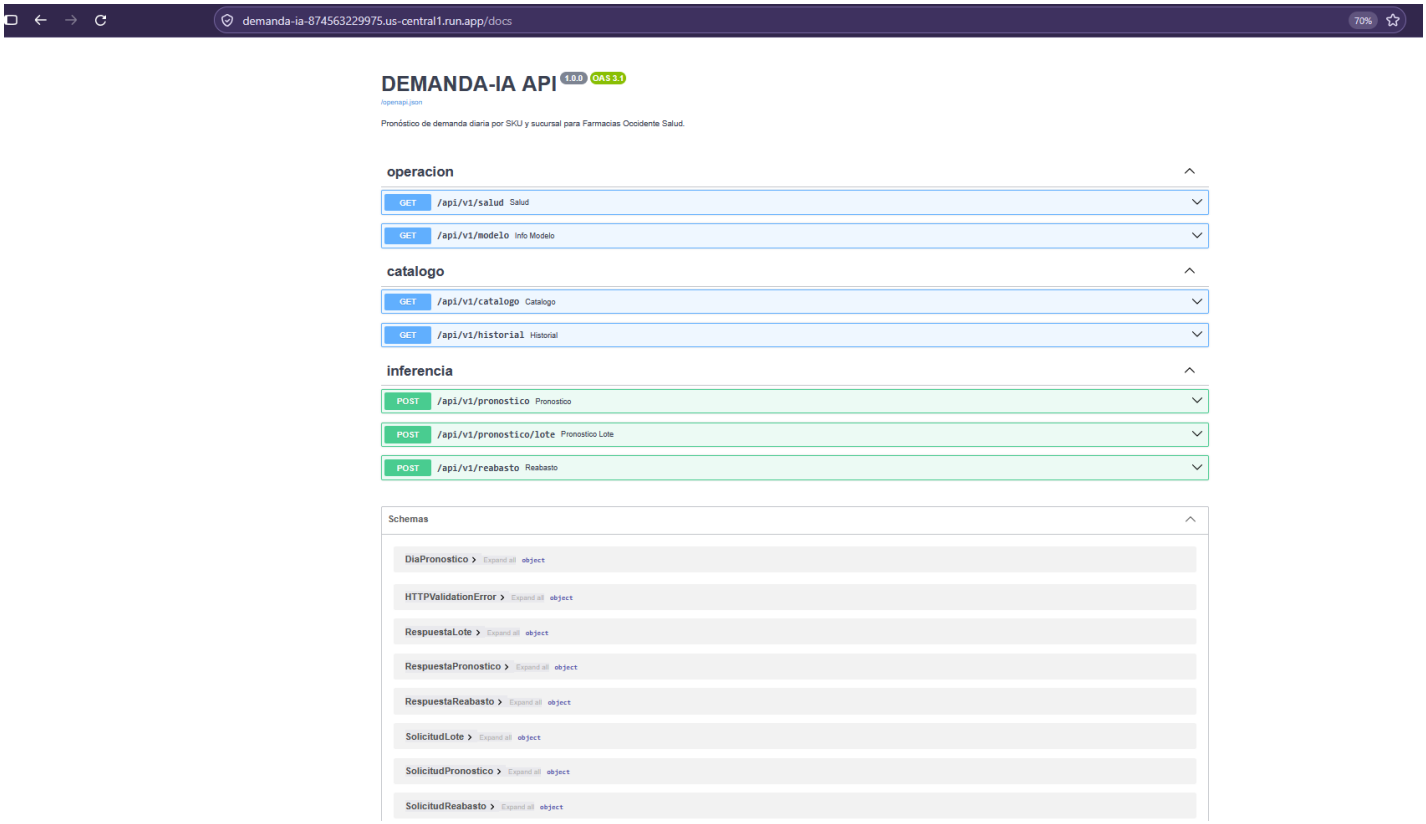


Figura 15. Documentación interactiva (Swagger) servida desde Cloud Run, con los endpoints agrupados en operación, catálogo e inferencia.

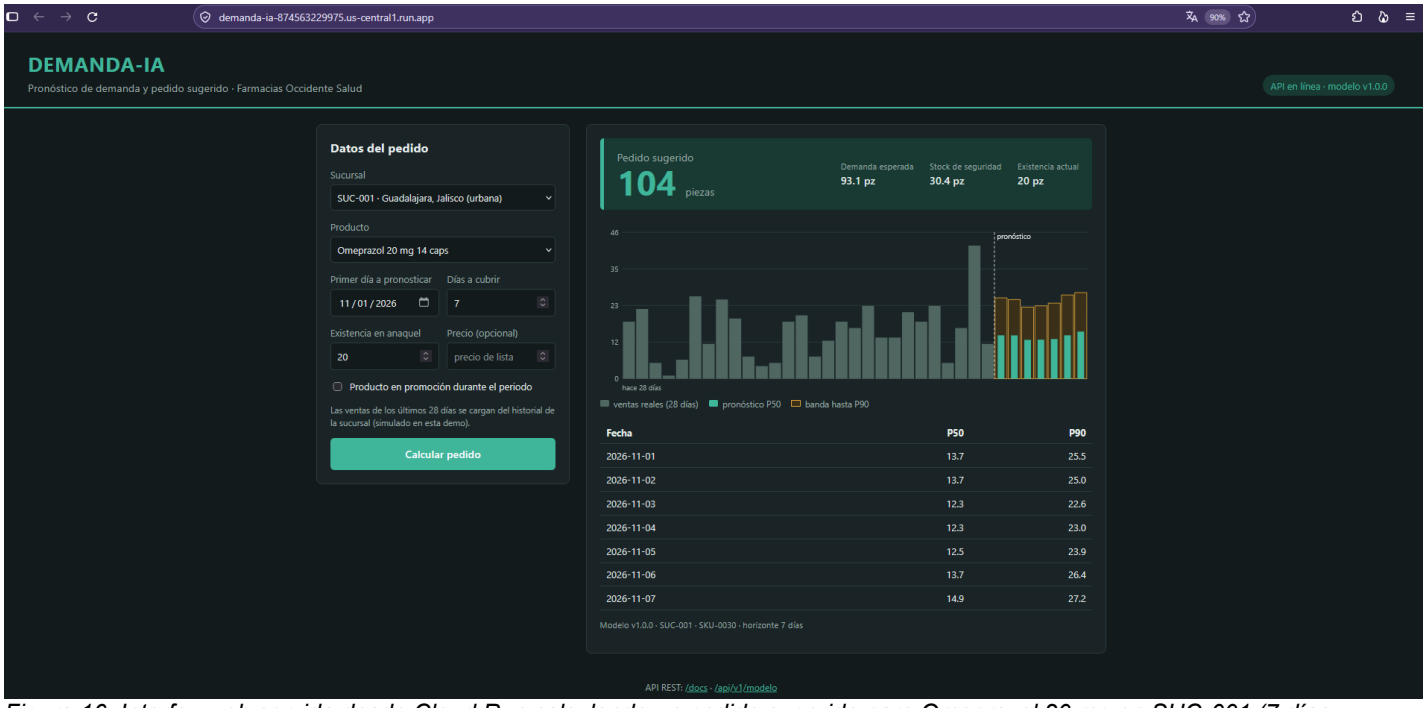


Figura 16. Interfaz web servida desde Cloud Run calculando un pedido sugerido para Omeprazol 20 mg en SUC-001 (7 días, existencia 20): 104 piezas.

Universidad Tecmilenio

Desempeño

Operación	Repeticiones	Mediana	Percentil 95	Máximo
POST /pronostico (1 registro)	200	9.6 ms	14.1 ms	16.4 ms
POST /pronostico/lote (1000 registros)	20	50.9 ms	97.8 ms	114.8 ms
POST /reabasto (14 días)	50	96.1 ms	135.5 ms	140.9 ms
Corrida nocturna: 4 200 pronósticos en 5 lotes	1	0.22 s en total		

Tabla 9. Latencia medida contra el servicio levantado (2 workers).

El reabasto tarda más que un pronóstico suelto porque es recursivo: calcula día por día y usa cada pronóstico como historial del día siguiente. Aun así, queda en una décima de segundo. La corrida nocturna completa de la red, que era la necesidad central del caso de negocio, cabe holgadamente en un segundo. El cuello de botella en producción no va a ser el modelo, sino extraer y escribir los datos del ERP.

9.- Hallazgos y acciones

Hallazgo	Impacto	Acción
El P50 subestima la suma semanal en 11 a 14 %	Pedido corto si se usara solo el P50	Mitigado: el reabasto suma stock de seguridad. Para v1.1, modelo de media con pérdida Poisson
El modelo no reacciona a precios por encima de lista	Subidas de precio no bajan el pronóstico	Mitigado: advertencia en la API. Pendiente: incluir aumentos de precio en el entrenamiento
Error diario alto en categorías de baja rotación	Pronóstico diario poco útil para esos SKUs	Evaluar pronóstico semanal para dermocosmética, vitaminas e infantil
Cobertura del P90 entre 87.5 % y 90.6 %	Ligeramente bajo lo nominal	Aceptable; monitorear y recalibrar si baja de 85 %
Los picos atípicos no se extrapolan	Protege contra errores de captura	Comportamiento deseado; advertencia para revisión manual
Datos sintéticos	Las métricas validan el método, no el desempeño real	Repetir esta validación con 12 meses de ventas del ERP antes de producción

Tabla 10. Hallazgos de la validación.

10.- Conclusiones

- **DEMANDA-IA** pasó las pruebas que se plantearon al inicio. El modelo es mejor que las reglas simples en todas las categorías y en sucursales que nunca vio. A nivel semanal reduce el error del pedido en 30 % frente a repetir la semana anterior y queda a menos de dos puntos del oráculo, que representa el error atribuible al ruido de los datos. La API respondió bien a todos los casos válidos, rechazó con un mensaje claro los 18 tipos de entrada inválida que se probaron y nunca devolvió un error interno ni un pronóstico negativo. El frontend y el backend se comunican como se diseñó, y el servicio aguanta la corrida nocturna de toda la red en menos de un segundo.
- Lo que más aprendí en esta validación es que una métrica aislada engaña. Un WAPE diario de 46 % parecía un mal modelo hasta que lo comparé con el oráculo, y un P50 bien calibrado parecía listo para usarse hasta que medí el sesgo de la suma semanal. También el análisis de

sensibilidad encontró algo que las métricas no mostraban: el modelo no sabe qué hacer con precios altos.

- Para pasar de proyecto académico a producción faltan tres cosas: repetir esta validación con ventas reales del ERP, agregar autenticación a la API y poner en marcha el monitoreo semanal de error que se propone en el manual de despliegue.

Anexo

#	Archivo	Prueba	Estado
1	test_api_funcional.py	test_salud	PASSED
2	test_api_funcional.py	test_metadatos_modelo	PASSED
3	test_api_funcional.py	test_catalogo_completo	PASSED
4	test_api_funcional.py	test_historial_28_dias	PASSED
5	test_api_funcional.py	test_pronostico_puntual	PASSED
6	test_api_funcional.py	test_pronostico_lote	PASSED
7	test_api_funcional.py	test_reabasto	PASSED
8	test_api_funcional.py	test_frontend_servido	PASSED
9	test_api_funcional.py	test_openapi_documenta_endpoints	PASSED
10	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio0-sucursal_id]	PASSED
11	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio1-sku]	PASSED
12	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio2-lag_7]	PASSED
13	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio3-media_28]	PASSED
14	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio4-fecha]	PASSED
15	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio5-fecha]	PASSED
16	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio6-precio]	PASSED
17	test_casos_extremos.py	test_entradas_invalidas_dan_422[cambio7-campo_extra]	PASSED
18	test_casos_extremos.py	test_campo_faltante	PASSED
19	test_casos_extremos.py	test_json_malformado	PASSED
20	test_casos_extremos.py	test_ids_en_minusculas_se_normalizan	PASSED
21	test_casos_extremos.py	test_historial_en_ceros	PASSED
22	test_casos_extremos.py	test_valores_atipicos_altos	PASSED
23	test_casos_extremos.py	test_precio_fuera_de_rango_se_acota[1.0]	PASSED
24	test_casos_extremos.py	test_precio_fuera_de_rango_se_acota[500.0]	PASSED
25	test_casos_extremos.py	test_lote_vacio_y_lote_excedido	PASSED
26	test_casos_extremos.py	test_lote_con_un_registro_invalido_rechaza_todo	PASSED
27	test_casos_extremos.py	test_horizonte_fuera_de_limite[0]	PASSED
28	test_casos_extremos.py	test_horizonte_fuera_de_limite[15]	PASSED
29	test_casos_extremos.py	test_historial_corto_o_negativo	PASSED
30	test_casos_extremos.py	test_existencia_suficiente_pedido_cero	PASSED
31	test_casos_extremos.py	test_horizonte_maximo_14_dias	PASSED
32	test_casos_extremos.py	test_historial_fuera_del_periodo_simulado	PASSED
33	test_casos_extremos.py	test_propiedad_p90_mayor_igual_p50_y_no_negativo	PASSED
34	test_modelo.py	test_variables_entrenamiento_igual_a_inferencia	PASSED
35	test_modelo.py	test_ventanas_historial	PASSED

36	test_modelo.py	test_simulacion_determinista_entre_rangos	PASSED
37	test_modelo.py	test_festivos_mexico_2026	PASSED
38	test_modelo.py	test_inferencia_determinista	PASSED
39	test_modelo.py	test_modelo_supera_baselines	PASSED
40	test_modelo.py	test_cobertura_p90_calibrada	PASSED
41	test_modelo.py	test_generaliza_a_sucursales_no_vistas	PASSED
42	test_modelo.py	test_promocion_aumenta_pronostico	PASSED
43	test_modelo.py	test_festivo_reduce_pronostico	PASSED
44	test_modelo.py	test_quincena_eleva_cronicos	PASSED
45	test_modelo.py	test_antigripales_suben_en_invierno	PASSED

Bibliografía

Chen, M. (2025). *¿Qué es una API (interfaz de programación de aplicaciones)? Oracle.* Obtenido de https://www.oracle.com/latam/cloud/cloud-native/api-management/what-is-api/

China, C. y Goodwin, M. (s.f.). *IaaS, PaaS, SaaS: ¿cuál es la diferencia? IBM.* Obtenido de https://www.ibm.com/mx-es/think/topics/iaas-paas-saas

Docker Inc. (s.f.). *Dockerfile reference.* Obtenido de https://docs.docker.com/reference/dockerfile/

FastAPI. (s.f.). *FastAPI in Containers - Docker*. Obtenido de https://fastapi.tiangolo.com/deployment/docker/

Google Cloud. (s.f.). *Cloud Run documentation.* Obtenido de https://cloud.google.com/run/docs

ISO/IEC. (2022). *ISO/IEC 23053:2022 Framework for Artificial Intelligence (AI) Systems Using Machine Learning (ML). International Organization for Standardization.*

Pedregosa, F., et al. (2011). *Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.*

Universidad Tecmilenio. . (2026). *Tema 15. Contenerizar el modelo con Docker [material de curso]*. Obtenido de https://cursos.tecmilenio.mx/courses/252869

Universidad Tecmilenio. (2026). *Tema 16. Despliegue en la nube [material de curso].* Obtenido de https://cursos.tecmilenio.mx/courses/252869